hydra_satsuma: Certified Structure Dispatch with a satsuma-iter+kissat Fall-Through

Greg Sidebottom

Research note — logic repo (https://github.com/gsidebottom/logic), SAT track, 2026-08-12. Solver description for the `hydra_satsuma` backend of `sat`: the certified `hydra` structure-dispatch pipeline (Cook-shape prover, XOR/GF(2) stage) whose fall-through engine is satsuma-iter+kissat, winner of the SAT Competition 2026 main track. Includes the completed three-way apples-to-apples comparison (`hydra` / `satsuma` / `hydra_satsuma`) on the official 2026 benchmark set, all on one Mac mini (M4 Pro, 64 GB; §2.1).

Abstract

`hydra_satsuma` is a structure-dispatch SAT solver: before any CDCL search, it tests the input for algebraic structure that admits a *polynomial-size certificate*, and only when no structure applies does it hand the formula to a general-purpose engine. Two structural stages run in order: (1) a *Cook-shape prover* that recognizes five cardinality-contradiction families (pigeonhole and its generalizations) and emits the Cook-1976 extension-variable refutation as a VeriPB proof — families on which resolution, and hence any DRAT-based route without new variables, is exponential; (2) an *XOR/GF(2) stage* that recovers parity constraints from their clausal encodings and runs Gaussian elimination, certifying parity refutations in VeriPB by Gocht–Nordström's division-based method (GN21). The fall-through engine is satsuma-iter+kissat (Anders et al.), the SAT Competition 2026 main-track winner, run unmodified in a container: satsuma iterates simplify–order–break rounds, adding only unit and binary breaking clauses, and emits a binary substitution-redundancy (SR) proof justifying each addition, kissat appends its clausal refutation to the same file, and `dsr-trim` verifies the *composed* proof against the exact CNF handed over — so symmetry-broken UNSAT results remain checker-certified. Every UNSAT path thus produces a machine-checked certificate in a format on the SAT Competition 2026 checker menu (VeriPB) or in the winner's own SR chain; SAT results carry the model as witness, projected to the original variables.

1 Pipeline

The backend runs the following stages in order; the first stage to reach a verdict ends the run. Stage budgets and caps are given with each stage.

1.1 Cook-shape prover

A syntactic detector (inputs up to 10^6 clauses) recognizes five structured UNSAT families, reading the exact clause layout rather than solving:

- **PHP-n-m** — the canonical pigeonhole formula, n pigeons, m holes, $n > m$;
- **RoundRobin** — sports-scheduling infeasibility, a per-pair/per-day two-level pigeonhole;
- **Embedded pigeonhole** — P disjoint at-least-one clauses over S complete slot-mutex cliques with $P > S$, the generalization covering e.g. MVRoundRobin;
- **Clique-coloring** — a graph asserted to contain a K-clique yet be C-colorable with $K > C$, proved by composing the clique-membership and coloring layers;
- **Mutilated chessboard** — bipartite perfect-matching infeasibility with imbalanced sides, proved by the two-sided cardinality argument.

On a match the verdict is UNSAT *by construction*, in microseconds to milliseconds, and — when a proof is requested — the module emits the corresponding pseudo-Boolean refutation in VeriPB format. For PHP the proof is exactly Cook's 1976 construction: fresh variables $Q_{ij} = P_{ij} \vee (P_{im} \wedge P_{nj})$ encode a pigeonhole instance with one fewer pigeon and hole; the reduction iterates down to a trivially refutable base. Each layer is polynomial, rendered as VeriPB `red` steps with the definition as witness. This matters because Haken's 1985 lower bound makes PHP exponential for resolution: a solver-emitted DRAT proof (which cannot introduce variables the solver never used) is out of reach at scale, while the extension-variable route is short, and VeriPB is one of the three checkers on the SAT Competition 2026 menu (§2.2).

1.2 XOR/GF(2) stage

For inputs up to 5×10^6 clauses, the stage recovers XOR constraints from their CNF encodings (complete canonical encodings only; the certifier re-derives the recovery strictly, so no trusted parsing) and runs GF(2) Gaussian elimination. Three outcomes:

- **Inconsistent system** (by unit propagation or by GE): the formula is UNSAT. When a proof is requested the stage emits a parity refutation following Gocht and Nordström's method [4] (henceforth **GN21**): parity reasoning, though exponential for resolution, has short pseudo-Boolean certificates, because cutting planes' *division* rule captures mod-2 arithmetic — encode each recovered XOR as pseudo-Boolean constraints, and summing two of them followed by division by two derives their GF(2) sum in a constant number of proof steps. Our certificate names the (typically small) refuting subset of recovered XORs and derives the contradiction $0 \geq 1$ from their sum, checkable by `veripb` in milliseconds. (The certifier re-derives the XOR recovery itself from the clausal encodings, so the proof does not trust the recovery code.)
- **Pure-XOR satisfiable**: GE solves the system outright; the model is the certificate.
- **Partial simplification**: GE forces some variables and simplifies the clause set, yielding an equisatisfiable *residual*. What happens next depends on *certified mode* (§1.3).

1.3 Certified mode

The `--proof` flag is the certified-mode switch for the whole hydra family. The subtlety is the partial-simplification case: an engine refutation of the *residual* does not certify the *original* formula — the GE step connecting them is real reasoning that the downstream proof does not contain. Therefore:

- **Certified mode** (`--proof` present; the benchmark harness always passes it): the GE simplification is *discarded* and the engine solves the original formula. Full end-to-end certification beats the simplification — a trade measured to be nearly free (§3).
- **Uncertified mode** (no `--proof`): the engine solves the residual. Verdicts remain sound (GE derives only logical consequences; SAT models are reconstructed by overwriting GE-forced variables), but a residual-path UNSAT is reported explicitly as uncertified for the original.

Emitting a proof *of the GE step itself* (deriving the forced units and simplified clauses via extension variables, in DSR or VeriPB) would close this gap and is well-understood machinery; it is deliberately deferred because measurement shows nothing is currently lost (§3).

1.4 Fall-through: satsuma-iter+kissat

When no structural stage decides the instance, `hydra_satsuma` hands it to the SAT Competition 2026 main-track winner, built unmodified from the official competition tarball (Anders et al.) and run in an Ubuntu 24.04 container:

1. `satsuma fix <cnf> --bsr --proof-file proof.out --out-file sb.cnf` — the Simplify–Order–Break–Repeat algorithm [9] (Best Paper, SAT 2026): iterate *simplify* (CNF preprocessing), *order* (clique-guided variable ordering via the CLIQUER maximum-clique solver on the variable–clause graph), and *break* (adding *only unit and binary* symmetry-breaking clauses, generated systematically along a

Schreier–Sims pointwise-stabilizer chain), until only trivial symmetries remain. Because breaking adds only units and binaries, simplification can shrink the formula — often below the input size — and expose new symmetries for the next round. The emitted *binary SR proof* justifies each breaking clause with its symmetry as substitution witness;

2. `kissat sb.cnf proof.out` — the CDCL engine solves the broken formula and *appends* its clausal refutation to the same proof file;

3. on UNSAT, `dsr-trim -f <cnf> proof.out` verifies the *composed* proof — satsuma's SR prefix plus kissat's refutation — against the exact CNF handed over.

The composition is the point: symmetry-breaking clauses are not consequences of the formula (they are satisfiability-preserving, not equivalence-preserving), so a bare engine refutation of the broken formula certifies nothing about the input. SR's substitution witnesses make the breaking steps checkable, and because kissat's clause additions are valid SR steps, one checker validates the whole file. SAT models over satsuma's augmented variable space are projected back to the original variables (sound: breaking clauses only remove models).

Operationally the container runs are bounded end to end:

- **Solve phase**: in-container `timeout` at the remaining budget minus 2 s, so the container always self-terminates and is never orphaned by the host-side watchdog. The verdict and solve time are recorded *before* any verification.

- **Verify phase**: a second bounded container run (`--satsuma-verify-secs`, default equal to the solve timeout, matching the harness's proof-check budget convention for the LRAT chain; 0 = unlimited). A verification timeout downgrades the row to sound-but-unchecked; the verdict is never lost.

- **Memory**: optional per-container hard cap (`--satsuma-mem-gb`); an instance exceeding it fails alone.

- **Size guard**: instances with $\geq 10^6$ variables or $\geq 2.5 \times 10^7$ clauses skip satsuma (its literal graph on such inputs is several GB) and run kissat directly on the raw formula; an UNSAT proof remains dsr-trim-checkable, since kissat's steps are the suffix of every composed proof.

The sibling backends complete the family: `hydra` (same structural stages, CaDiCaL fall-through with native LRAT checked by `cake_lpr`) and `satsuma` (the winner bare, no structural stages — the baseline arm for measuring what the dispatch adds).

2 Certificates by path

Path	Verdict	Certificate	Checker
Cook shape	UNSAT	Cook-1976 extension proof (VeriPB)	`veripb`
XOR inconsistent	UNSAT	GN21 parity proof (VeriPB)	`veripb`
Pure-XOR solvable	SAT	model witness	—
satsuma+kissat	UNSAT	composed binary SR	`dsr-trim`
satsuma+kissat	SAT	model (projected)	—
GE residual (uncert. mode)	UNSAT	none for the original	—

Verification of the SR path happens *in-solver* (the second container phase), and the harness credits it from the solver's `dsr-trim VERIFIED UNSAT` marker rather than re-checking — binary SR is not readable by the LRAT/GRAT/VeriPB checkers, and routing it to one of them would produce a spurious rejection.

2.1 Checker cost: hinted vs. unhinted

The two verification chains do structurally different work. `cake_lpr` consumes LRAT, where every step carries unit-propagation hints computed by CaDiCaL at solve time; checking is a near-linear replay. `dsr-trim` consumes unhinted DSR — like `drat-trim`, it must find every propagation itself and discharge each SR step's redundancy goals, i.e. it performs the elaboration that the LRAT path amortized into the solve. A same-CNF measurement on the evaluation machine — an Apple Mac mini with the M4 Pro chip (10 performance + 4 efficiency cores), 64 GB RAM, macOS 26, the satsuma toolchain in an Ubuntu 24.04 Docker VM; every run in this note uses it — (instance `b20`, ISCAS circuit, UNSAT):

Chain	Proof	Size	Check time
CaDiCaL native LRAT	hinted	40 MB	`cake_lpr` 1.55 s
satsuma+kissat DSR	unhinted	20 MB	`dsr-trim` 7.8 s

SR's compensation is succinctness (the same instance's SR proof is half the LRAT's size; Codel–Avigad–Heule report SR proofs at 0.4% of the line count of their RAT translations) and expressiveness (the symmetry steps have no DRAT analogue without extension variables). Memory behaves oppositely: `dsr-trim` is a compact C program (no out-of-memory events across concurrent checking in our runs), while `cake_lpr`'s CakeML runtime requires a pre-sized heap that scales with the proof and is the component our harness bounds with a dedicated memory cap and a concurrency semaphore.

2.2 Competition-format compliance

SAT Competition 2026 accepts three proof checkers in the main track: DPR (`dpr-trim`), GRAT (`gratgen`), and VeriPB (`veripb`). The Cook and GN21 certificates are VeriPB proofs and verify under the same `veripb` the competition runs; the SR chain is the 2026 winner's own. One gap separates `hydra_satsuma` from literal entry shape: the rules have a solver declare a *single* checker and write one `proof.out`, whereas this backend emits per-path formats. Unification is mechanical future work — either translate the engine's clausal refutations into VeriPB (`rup`/`del` steps map directly; RAT additions become `red` with witness) and declare VeriPB for everything, or render the Cook/GN21 constructions in DRAT-with-extension-variables and declare the clausal chain. The VeriPB route is helped by satsuma itself: its SR steps can be emitted in either DSR or VeriPB format [9], so only the engine refutation needs translating.

3 Measured design decisions

Two measurements on the official 2026 main-track set (400 instances with duplicates; the evaluation machine) fixed design choices in advance of the full performance run:

- **GE simplification is worth ≈ nothing on this set.** Every instance on which the partial-simplification path fires (twelve, all ISCAS-class circuits) recovers exactly one XOR and forces exactly one variable — on formulas of 2×10^3 to 3.4×10^5 variables. Solve-only A/B (original vs. residual, same engine): `b20` 7.43 s vs. 7.40 s, `s38417` 1.66 s vs. 1.68 s, `c7552` 0.75 s vs. 0.77 s — within noise. Certified mode's "ignore GE, solve the original" therefore costs nothing measurable, and building the GE-step certifier (§1.3) is not currently justified by performance.
- **Where the XOR stage does pay, it is already certified.** In the prior full `hydra` run on this set, eleven instances were decided outright by the parity stage in ∼10 ms each, all eleven with verified GN21 certificates; twenty-seven more fell to the Cook prover with verified VeriPB proofs.

4 The benchmark set: construction and comparability

4.1 Construction

The evaluation set is the official SAT Competition 2026 main-track selection, reconstructed as follows and auditable at every step:

- **Source of truth**: the competition's benchmark-compilation-script tarball (https://satcompetition.github.io/2026/downloads/) carries the authoritative `selected_benchmarks.csv` with columns `(isohash2, hash, author, family, result)`. The file is archived in-repo at `tools/gbd/sc2026_selected_benchmarks.csv`.

- **Slot count and duplicates**: the CSV has exactly **400 rows over 391 distinct hashes**: nine hashes each appear twice, *in the official selection itself* (the ISCAS circuit `b20`, hash `c4318c6a…`, is one of them). Re-verified from the archived CSV and from the built index at the time of writing.
- **Index**: `main_track_2026_official.jsonl` is built row-for-row from the CSV (400 records), each carrying the hash, the original filename, the family/author metadata, and the competition's expected result where known.
- **Instance identity**: instance files are hash-addressed (`<hash>-<name>.cnf.xz`) and were fetched from the GBD benchmark database *by* those hashes; the GBD hash is the MD5 of the normalized formula (not of the file bytes), so agreement between the official CSV and the on-disk files is content-level identity under GBD normalization, not filename trust.
- **Dedup semantics**: the harness by default deduplicates to the 391 unique instances and prints `deduped 9 duplicate records (kept 391 unique)`; `--no-dedupe` runs all 400 slots exactly as officially scored (a duplicated instance counts twice).
- **Result integrity**: every row is cross-checked against the competition's expected status where recorded (any SAT/UNSAT disagreement is flagged as a mismatch; none has been observed), SAT verdicts carry validated witnesses, and UNSAT verdicts carry the per-chain certificates of §2.

4.2 Comparability with the official 2026 standings

Absolute solved counts measured here must *not* be read against the official competition standings. The official main-track winner, satsuma-iter+kissat, scored **276/400** (PAR-2 3647.02) under 5000 s CPU per instance, one instance per node, on the competition's cluster hardware. Our protocol runs 5000 s *wall-clock* per instance with *ten instances concurrently* on one 14-core Apple Silicon machine (64 GB host; the satsuma-based arms execute inside a ∼47 GB Linux VM). Per-core, per-second compute on the M4 Pro substantially exceeds the competition nodes'. For reference:

Single-thread benchmark	Apple M4 Pro	Xeon E3-1230 v5
PassMark ST	4506	2189
Geekbench 6 ratio	≈ 2.9×	

The evaluation machine is the Mac mini described in §2.1. The competition node model is *assumed*, not confirmed: the 2026 results slides name no hardware (they do confirm CPU-time scoring), but the infrastructure is Beyer's LMU cluster — the winning submission's own archive carries that cluster's competition-environment Dockerfile — whose documented SV-COMP node is an Intel Xeon E3-1230 v5 @ 3.4 GHz (Skylake, 2015). Published single-thread figures put the M4 Pro P-core at roughly **2–3**× that core (PassMark 4506/2189 ≈ 2.1×; Geekbench 6 ≈ 2.9×). Two effects pull the *effective* per-instance ratio below the raw single-thread number: our ten concurrent workers share one machine's memory bandwidth, whereas the competition dedicates a node per instance; and CPU-second vs. wall-second accounting differs. Equal nominal seconds still buy roughly twice the search here, which on heavy-tailed runtimes pulls many near-timeout instances under the line — and is why local solve counts (e.g. the `hydra` baseline's 311 of the 391 unique instances, 318 of the 400 official slots) run well above the official 276/400 despite the contention working in the opposite direction. The bare `satsuma` arm turns this from estimate into measurement: it is the winner's own pipeline, built unmodified from the official submission archive, on the official instance set — so its local 334/400 against the official 276/400 is a pure environment delta. Reading it off the cactus curve: local `satsuma` reaches 276 solves at a per-instance budget of only **653 s**, where the competition needed its full 5000 s (CPU) for the same count — an effective environment factor of 5000/653 ≈ **7.7**×, and conservative in direction, since our ten-way contention slows the local arm while competition instances ran on dedicated nodes. (The estimate is count-based, not instance-matched: official per-instance results are not published alongside the standings.) Notably, the single-thread microbenchmark ratios above (2–3×) substantially *underpredict* the measured end-to-end factor: CDCL search is memory-latency-bound, and the M4 Pro's memory subsystem advantage over a 2015 Skylake-E3 core far exceeds its arithmetic advantage — a reminder that microbenchmark constants do not price real workloads. Local solve counts therefore overshoot official standings mechanically, by roughly this factor's worth of extra effective budget per instance. One axis runs the *other* way: the competition checks proofs under a 45,000 s budget per certificate (9× the solve timeout; stated for SC2020 and SC2025, the standing convention), whereas our protocol checks under 5,000 s (plus a 16 GB heap cap for `cake_lpr`) — so local *certification* counts are conservative relative to competition conditions: a proof recorded here as unchecked-on-timeout might well verify under the official budget. The comparisons this note will report (§5) are therefore *internal*: all three arms under one fixed protocol on one machine, where hardware and scheduling effects cancel and only the solver pipelines differ. The official figures are quoted for context only.

5 Results

All three arms have completed on the official 400-slot set (5000 s per instance, certified mode, ten parallel workers, the evaluation machine; satsuma arms in Linux/Docker). All numbers are internal to the protocol and caveats of §4.2; soundness was clean across all three runs (zero expected-status mismatches, zero witness failures, zero contradictory verdicts between any pair of arms).

Arm	Solved/400	SAT	UNSAT	PAR-2	UNSAT certified
`hydra`	318	132	186	2459	184/186
`satsuma`	334	139	195	2015	177/195
`hydra_satsuma`	**335**	139	196	**1988**	170/196

Both completed runs use the identical protocol on all 400 official slots (the selection's nine duplicate hashes each occupy two slots and count twice, as in official scoring). `hydra`'s two unchecked proofs are `cake_lpr` memouts; the satsuma-chain unchecked proofs are `dsr-trim` timeouts (§5.2).

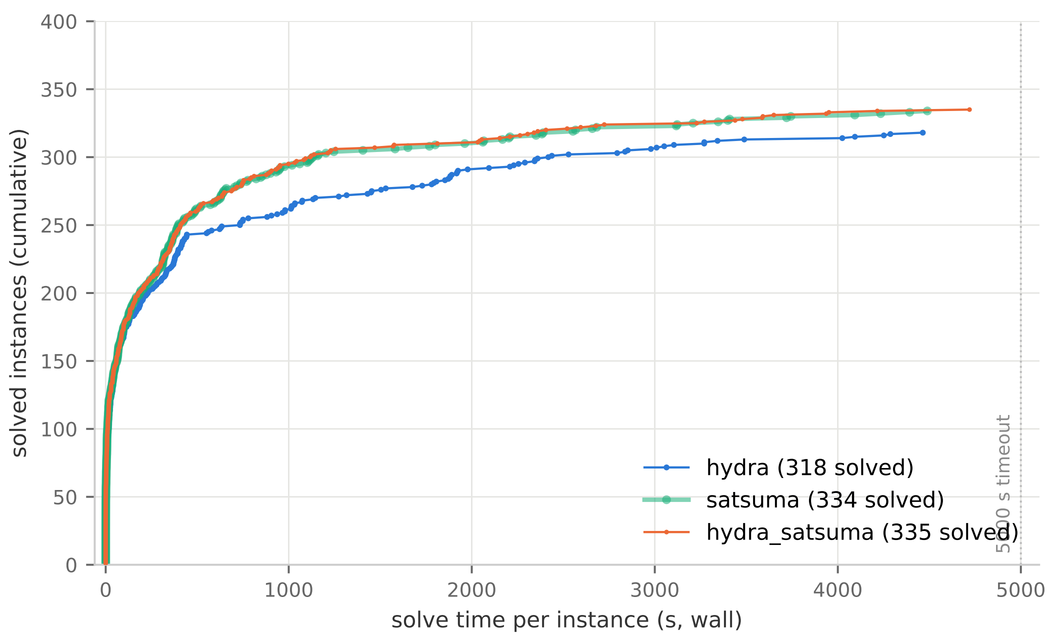

Figure 1: Cactus plot on the official 400-slot set: cumulative instances solved within a given per-instance time budget; one dot per solved instance. `hydra` is aligned to the official slots per hash. `satsuma` is drawn as a wide translucent band beneath `hydra_satsuma`'s thin line because the two curves coincide almost everywhere — that near-total overlap is itself the finding (§5.1): with a symmetry-capable engine, the structural front-end changes the curve only at the chessboard pair, while the engine swap separates both from `hydra` from ∼250 instances on.

Engine effect (`hydra` vs. `hydra_satsuma`, structure held fixed): replacing CaDiCaL by satsuma-iter+kissat is worth +17 slots and −19% PAR-2. On the 391 unique instances the satsuma arm solves eighteen that `hydra` does not — concentrated in symmetric design families (both `SC25_Timetable` instances, the `lockchart` families, `ramsey_4_4_18`, `oddball`, a `le450` coloring, several multiplier-equivalence UNSATs) — against three in the other direction with no structural signature.

5.1 What the structural front-end is worth

The `satsuma` vs. `hydra_satsuma` pair isolates the Cook/XOR machinery exactly: same engine, same container, same protocol; the only difference is the structural front-end. Across the 362 slots where no structure applies the two arms' solved sets are *identical* (zero flips in either direction); every count and score difference between them traces to the 38 structural slots plus measurement noise.

Benefit where it applies (38 slots: 27 Cook, 11 GN21): `hydra_satsuma` decides all 38 in **0.68 s total**, every verdict carrying a verified VeriPB certificate. Bare `satsuma` solves 37 of 38 at a PAR-2 cost of 14,431 s: `mchess_20` times out (the run's only count flip, worth 25 of the 27-point final PAR-2 gap), `mchess_17` barely lands at 4392.7 s — 88% of budget, and its SR proof check then times out, so under official only-verified-counts semantics the solve would not score — and `tseitingrid6x185_shuffled`, a Tseitin parity contradiction, takes 14.1 s against the GN21 path's 0.025 s. The honest fine print: 35 of the 38 are easy for the winner too (median 0.4 s), so *nearly the whole structural credit concentrates in the mutilated-chessboard pair* — the one family in this set that is resolution-exponential, not rescued by SOBR-style unit/binary symmetry breaking, and trivial for the two-sided cardinality argument. The front-end's value against a symmetry-capable engine is thus a *hard floor under structured families*, plus instant polynomial certificates, rather than a broad speedup: on a set without such families it would contribute certification and latency only; on a set with more of them (larger chessboards, larger Tseitin grids — both grow without bound) the count gap widens mechanically.

Overhead where it does not apply (362 slots): measured as the median paired solve-time difference on the 297 instances both arms solved — same formula, same engine, so the pairing isolates the detection prefix — the cost is **+1.16 s per instance**, i.e. ≈ 420 s over the whole run, or **1.05 PAR-2 points**. (The paired-difference *sum* is larger, +3684 s, but the excess beyond the median-based prefix sits in a handful of multi-thousand-second solves stretching 5–13% — proportional to solve time, the wrong shape for a fixed prefix, and consistent with wall-clock load variance between runs; kissat's search on identical input is deterministic.) Detection never caused a flip: no instance was lost to overhead.

The ledger closes: +36.1 PAR-2 points of structural credit against ≈ 1 point of measured prefix (and ≈ 8 of tail noise) nets the observed 27-point, one-solve advantage — a ∼34:1 benefit-to-overhead ratio on the honest fixed-cost accounting, delivered almost entirely by one family.

5.2 Certification ledger

Scores above count solved verdicts; they are *not* adjusted for local proof-checking timeouts, since our checking budget (5000 s) is 9× tighter than the official 45,000 s (§4.2). Per arm: `hydra` certified 184/186 UNSATs (2 `cake_lpr` memouts under the local 16 GB heap cap); `hydra_satsuma` 170/196 (26 `dsr-trim` timeouts, which additionally received less than nominal budget due to a since-fixed process-wall bug); `satsuma` 177/195 (18 `dsr-trim` timeouts — fewer than `hydra_satsuma` despite near-identical workload because this run included the wall fix, so its verify budgets were fully usable). No proof was *rejected* anywhere; every gap is a resource limit. The 38 structural certificates and the SAT witnesses never strain any budget. We assume the `dsr-trim` timeouts complete within the official 45,000 s: the competition counts an instance only if its proof checks, SR was an official 2026 checker, and the winner's official 276/400 is therefore net of SR verification at that budget on competition hardware (`custmulsb2x32o`, unchecked in both satsuma-chain runs here, is the strongest candidate to need the full official budget). Offline re-checks at 45,000 s are prepared for both the 25 unique `hydra_satsuma` instances and `hydra`'s two memouts (48 GB heap).

Reading. The three-way resolves cleanly. The engine is worth +16/+17 slots over CaDiCaL on this set; the structural front-end is worth +1 slot and 27 PAR-2 points against the strongest available engine — all of it from one resolution-exponential family — at a measured cost of ≈ 1 PAR-2 point of detection prefix and zero lost instances. Its deeper contributions are invariants the count does not show: every structural verdict is instantaneous and carries a polynomial VeriPB certificate (against engine-time SR proofs that are expensive to check and, twice here, uncheckable in budget), and the floor it provides is unbounded in the direction the engine's weakness grows.

Acknowledgments

Claude (Fable 5, Anthropic) assisted with the implementation, measurement, and drafting throughout. satsuma and dejavu are by Markus Anders et al.; kissat by Armin Biere; `dsr-trim` and `lsr-check` by Cayden Codel; `cake_lpr` by Yong Kiam Tan, Marijn Heule, and Magnus Myreen; VeriPB by Stephan Gocht, Jakob Nordström, et al. The satsuma-iter+kissat components are built unmodified from the official SAT Competition 2026 submission archive and run as-is in a container.

References

[1] S. A. Cook. The complexity of theorem-proving procedures. *STOC*, 1971.

[2] S. A. Cook. A short proof of the pigeon hole principle using extended resolution. *SIGACT News*, 8(4):28–32, 1976.

[3] A. Haken. The intractability of resolution. *Theoretical Computer Science*, 39:297–308, 1985.

[4] S. Gocht and J. Nordström. Certifying parity reasoning efficiently using pseudo-Boolean proofs. *AAAI*, 2021.

[5] C. R. Codel, J. Avigad, and M. J. H. Heule. Verified substitution redundancy checking. *FMCAD*, 2024. https://www.cs.cmu.edu/~mheule/publications/SRcheck.pdf

[6] Y. K. Tan, M. J. H. Heule, and M. O. Myreen. `cake_lpr`: Verified propagation redundancy checking in CakeML. *TACAS*, 2021.

[7] F. Pollitt, M. Fleury, and A. Biere. Faster LRAT checking than solving with CaDiCaL. *SAT*, 2023.

[8] SAT Competition 2026: certificates and checkers. https://satcompetition.github.io/2026/output.html; winner submission archive https://satcompetition.github.io/2026/downloads/solvers/anders.tar.xz.

[9] M. Anders, C. Codel, and M. J. H. Heule. Simplify, order, break, repeat. *SAT 2026*, LIPIcs vol. 377, article 4, pp. 4:1–4:19. DOI `10.4230/LIPIcs.SAT.2026.4`. Best Paper Award, SAT 2026. Software: DOI `10.5281/zenodo.20185023`; satsuma: https://github.com/markusa4/satsuma.